DynaByte: Shared-Budget Allocation of Expert Precision, Residency, and Lookahead

ANONYMOUS AUTHOR(S)

On one GPU, the bytes left after non-expert weights, the KV cache, and activations are one expert-memory budget, and the host-to-device link is one copy queue. A byte in that budget pays for a larger high-precision copy, for keeping an expert on the GPU, or for starting a copy early enough that the transfer overlaps compute. A byte spent on one of these cannot do the other two, and fixing two while tuning the third can still raise tail latency. DynaByte admits a requested high-precision copy, resident expert, or early copy only together with the experts it demotes or evicts to free those bytes. That exchange is admitted only when the release lowers predicted exposed wait and $\widehat{R} \le \widehat{\epsilon}/\kappa$. $\kappa > 1$ is fixed on the calibration prompts. A copy becomes executable only after it finishes, and the expert-memory peak stays within budget. A state is published only after a side probe measures $\Delta\text{PPL} \le \epsilon$. The probe in Algorithm 1 runs on the serving GPU after the candidate copy is reserved inside B, and on these streams it finishes before the timed requests. On the three evaluation streams the published states are the DynaByte rows of Table 5, 0.62, 0.52, and 0.82, and no exchange is rolled back. Its ΔPPL on the mixed evaluation prompts is 0.62, 0.52, and 0.82 against the all-high reference. The bounds fixed from Section 2.5 are $\epsilon = 0.70$, 0.60, and 0.90. The values 0.62, 0.52, and 0.82 are within those bounds. On the five downstream tasks, weighted by sample size, its averages are 73.49, 79.04, and 60.73 against the all-high averages 74.37, 79.55, and 61.36, lower by 0.88, 0.51, and 0.63 points (1.18%, 0.64%, and 1.03%) on Qwen3-30B, Qwen3-Next-80B, and Phi-3.5-MoE. Under the same expert-memory budget, it reduces p95 TPOT by 11.6% relative to HOBBIT on Qwen3-30B. On Qwen3-30B, Static-mixed reaches 43.7 ms at ΔPPL 0.79, 0.09 above $\epsilon = 0.70$.

CCS Concepts: • **Computing methodologies** → *Artificial intelligence*; • **Software and its engineering** → **Memory management**; • **Computer systems organization** → *Architectures.*

Additional Key Words and Phrases: mixture of experts, memory management, prefetching, quantization, performance modeling

ACM Reference Format:
Anonymous Author(s). 2027. DynaByte: Shared-Budget Allocation of Expert Precision, Residency, and Lookahead. *Proc. ACM Meas. Anal. Comput. Syst.* 1, 1 (October 2027), 28 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

A Mixture-of-Experts (MoE) layer routes each token to a small subset of experts, so capacity can grow without a proportional increase in per-token computation [15, 29, 37]. The expert weights still have to be stored or fetched. Models from Mixtral and Qwen3 to DeepSeek-V2 and GLaM keep tens to hundreds of billions of parameters, and in the largest case a trillion, while activating only a fraction of them per token [8, 10, 23, 47]. On one GPU those weights compete with non-expert parameters, the KV cache, activations, and runtime workspaces. When the experts cannot all remain at the desired precision, the system has to compress them and move them across the host-to-device link.

How should one GPU spend its expert bytes and its host-to-device link so that Mixture-of-Experts inference lowers tail latency, keeps measured ΔPPL within ϵ, and never holds more expert bytes than the budget?

Quantization fixes each expert's precision before serving [17, 30]. Mixed-precision methods choose that precision under a memory or accuracy target [12], and expert caches and prefetchers

2027. 2476-1249/2027/10-ART $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

then decide which experts stay on the GPU and when a missing expert is copied, usually for one fixed precision [14, 40, 46]. HOBBIT and DyMoE already run the low-precision copy, the cache, and the prefetch in one system [20, 42].

Four measurements show why a fixed split of those bytes fails. Perplexity rises as more experts move to the smaller, low-precision copy, so keeping an expert at high precision both spends bytes and makes a later transfer larger. WikiText, GSM8K, and HumanEval at the same layer call different experts, so a precision map or a resident set taken from one task misses the next. In prefill the bytes that can overlap compute grow with the batch while the selected bytes stay nearly flat; in decode the selected bytes grow and the overlap window does not. Raising the high-precision share from 25% to 40% also changes how many experts can stay on the GPU and how many layers ahead a copy can be staged. Section 2 reports these measurements. Section 3 spends the same bytes by demoting or evicting experts already on the GPU and using the freed bytes for the copy it is about to admit.

We make three contributions.

- We measure one expert-memory budget and one host-to-device queue with three spends: a larger high-precision copy, keeping an expert on the GPU, and copying an expert before it is used (Section 2). A fixed split cannot compare these spends when the routed experts and the prefill–decode mix change, and systems that already quantize, cache, and prefetch still decide the three spends separately.
- We design DynaByte to demote or evict experts already on the GPU to free bytes for the copy it admits, as specified in Section 3. The executor holds every transition's expert-memory peak within B. Admission requires $\widehat{R} \le \widehat{\epsilon}/\kappa$, with $\kappa > 1$ fixed on the calibration prompts. A state is published only when a side probe measures $\Delta\mathrm{PPL} \le \epsilon$.
- We evaluate DynaByte on one GPU with Qwen3-30B, Qwen3-Next-80B, and Phi-3.5-MoE. Latency replays a fixed order of MMLU-Pro, GPQA, AIME25, GSM8K, and HumanEval, split by input/output length, at batch size 8. The feasibility cap is ΔPPL on the mixed evaluation prompts. Downstream scores use the same five tasks, weighted by item count. DynaByte's ΔPPL on those prompts is 0.62, 0.52, and 0.82. These three values are within $\epsilon = 0.70$, 0.60, and 0.90. Its averages on the five tasks are 73.49, 79.04, and 60.73 against the all-high averages 74.37, 79.55, and 61.36, lower by 0.88, 0.51, and 0.63 points (1.18%, 0.64%, and 1.03%) (Table 4). Under the same expert-memory budget, p95 TPOT is 11.6% lower than HOBBIT on Qwen3-30B, 23% lower on Qwen3-Next-80B, and 27.1% lower on Phi-3.5-MoE (Table 5). The lowest p95 centers in Table 5 are 43.7 ms, 117 ms, and 51.8 ms. The rows at those centers have ΔPPL above $\epsilon = 0.70$, 0.60, and 0.90. Under Poisson arrivals from a client on the same host, SLO goodput is 0.82 requests/s against HOBBIT's 0.69 on Qwen3-30B (Table 8).

2 BACKGROUND AND MOTIVATION

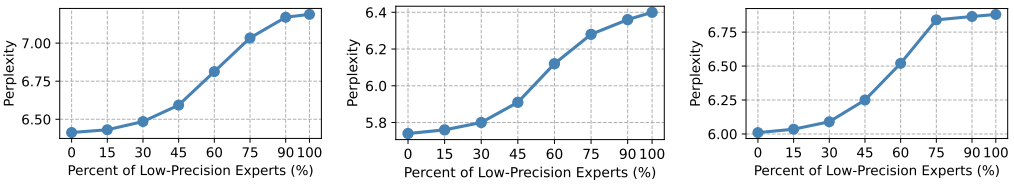
2.1 Expert Memory on One Accelerator

An MoE layer routes each token to k_ℓ experts, and the router determines which experts execute. Expert-choice routing lets each expert select its tokens [56]. That choice does not determine where their weights reside or when they arrive on the device. A selected expert that is absent must be transferred from host memory, and a demand transfer exposes that time on the forward path.

Memory for non-expert parameters, the KV cache, activations, and fixed workspaces is reserved first, and the remaining capacity is the expert-memory budget B. Host memory stores a packed low-precision representation and a packed high-precision representation of every routed expert, with aggregate footprints P_{lo} and P_{hi}. We write $\Phi = B/P_{\mathrm{lo}}$ and $\rho = P_{\mathrm{hi}}/P_{\mathrm{lo}}$. At any instant an expert is absent or resident at low or high precision, and the forward pass can use only a published resident

Table 1. Names used after this section. Precision denotes a stored tier, low or high.

Term	Meaning
Fidelity	Extra bytes of the high-precision representation, $\delta m_i^{\mathrm{fid}} = m_i^{\mathrm{hi}} - m_i^{\mathrm{lo}}$.
Lease	$z = (u, i, b, [t_s, t_f))$, with use $u \in \{\mathrm{fid}, \mathrm{res}, \mathrm{look}\}$.
Donor	A lease released to fund another lease.
Closure	Attaching a donor set D to a lease, forming the exchange $x = (Z^+, D, \mathcal{T})$.
Envelope S_t	Maximum routed-layer depth for candidate generation. It reserves no memory or bandwidth.
ϵ, $\widehat{\epsilon}$, κ	ϵ bounds measured ΔPPL. $\widehat{\epsilon}$ bounds $\widehat{R}$ before division by κ. Admission uses $\widehat{R} \le \widehat{\epsilon}/\kappa$, with $\kappa > 1$ fixed on the calibration prompts. Publication requires a side probe with measured $\Delta\mathrm{PPL} \le \epsilon$.



(a) Qwen3-30B: FP16/INT4 (b) Qwen3-Next-80B: INT4/INT2 (c) Phi-3.5-MoE: FP16/INT4

Fig. 1. WikiText-2 perplexity under frozen precision-quota sweeps. Each model is demoted along its own ranking, fixed before the sweep.

representation. The host-to-device copy queue has measured rate C. The size of the copy, whether the expert stays on the GPU, and when the transfer occupies the queue decide which transfers consume this rate.

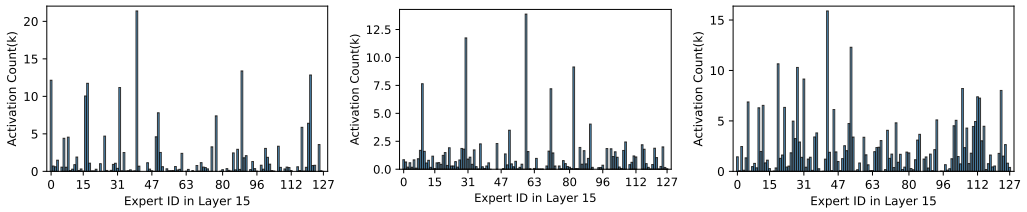
The same bytes in B pay for three things. *Fidelity* is the extra bytes of the high-precision copy. *Residency* keeps an expert on the GPU so a later use needs no copy. *Lookahead* starts that copy early enough to overlap earlier compute, and once the copy is published it is residency at the chosen tier. Table 1 lists these names.

2.2 A Larger Copy Costs Perplexity and Bytes

Figure 1 changes the fraction of experts stored at low precision. Each point demotes a longer suffix of one fixed ranking, so the curve is the perplexity cost of the low-precision fraction. The ranking stays frozen, and the sweep does not reassign which experts receive the higher-precision bytes.

WikiText-2 perplexity rises on Qwen3-30B (FP16/INT4), Qwen3-Next-80B (INT4/INT2), and Phi-3.5-MoE (FP16/INT4) as more experts move to the lower tier. Each model is demoted along its own ranking. The rise is continuous. Section 2.5 sets each model's ϵ to the smallest one-decimal number that covers that model's 25% high-precision gap. The higher-precision representation is also the larger packed copy. Raising the fidelity share therefore spends expert memory and increases the bytes a later transfer must move.

Perplexity in Table 4 is measured on the mixed evaluation prompts. On Qwen3-30B the all-low value there is 7.29, against 7.19 at the right edge of panel (a). On Phi-3.5-MoE it is 6.98, against 6.880 at the right edge of panel (c). Panel (c) starts at 6.010, this table's all-high value 6.01.



(a) WikiText (b) GSM8K (c) HumanEval

Fig. 2. Selected token–expert dispatch counts at Layer 15 of Qwen3-30B. All experts remain at low precision and the counters are reset between workloads.

2.3 Which Experts a Workload Uses Also Changes

Figure 2 counts which experts receive the bytes priced in Figure 1. It counts the token–expert dispatches at Layer 15 of Qwen3-30B on WikiText, GSM8K, and HumanEval. Every expert stays at low precision, and the counters reset between workloads, so the histograms show which experts each workload uses.

The three tasks dispatch different experts. A fidelity assignment or a resident set taken from one workload therefore misses experts the next workload uses. The counts record access frequency. They leave quantization sensitivity unmeasured, so they do not rank the quality harm of a low-precision expert.

2.4 Whether the Remaining Bytes Can Be Hidden Also Changes

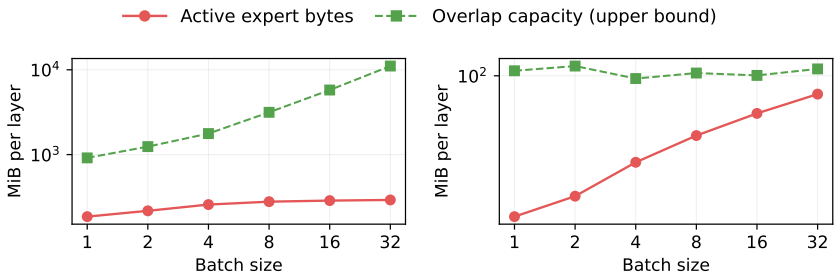
Figure 3 asks whether an expert that is still absent can arrive before it is used. It compares, for Qwen3-30B, the bytes selected in one MoE layer with an upper bound on the bytes that can move during that layer's compute.

Demand comes from five measured prompt blocks at each batch size. The bound multiplies measured blocking host-to-device bandwidth by mean TTFT or TPOT and divides by 48 routed layers. The bound is an optimistic ceiling: it ignores each expert's deadline and the bandwidth concurrent kernels take.

Prefill touches most routed experts, so selected bytes stay nearly flat while the overlap bound rises with the batch. Decode selects fewer bytes, and those bytes grow with the batch while the overlap bound stays near the per-layer decode window. In both panels the bound lies above selected bytes. The slack between them widens with batch size in prefill and narrows in decode, so one lookahead depth does not fit both phases. Representation size stays fixed in this comparison.

2.5 A Larger High-Precision Share Changes Who Stays and How Far Ahead the Copy Starts

Figure 4 places fidelity, residency, and lookahead on one expert-memory budget at the measured host-to-device bandwidth of the replay. The experiment replays a measured 2,048-token routing trace under the byte-accurate expert budget used by the Qwen3-30B runtime. Residents follow the frequency ranking from warm-up traces. At a 25% high-precision share, 12 staging slots leave 96 low-precision residents per layer and reduce exposed transfer to 0.44 ms/request. At a 40% high-precision share, exposed transfer is minimized at eight slots with 43 low-precision residents, then rises because additional staging displaces residents. Raising the high-precision share from 25% to 40% therefore changes both residency and the staging depth that reduces exposed transfer. On Figure 1(a) these two shares sit 0.62 and 0.40 above all-high, 7.034 − 6.413 and 6.813 − 6.413, and the 10% share and the all-low point sit 0.76 and 0.78 above it, 7.170 − 6.413 and 7.189 − 6.413. On



(a) Prefill (b) Decode

Fig. 3. Active expert bytes and an upper bound on overlap capacity for Qwen3-30B. The bound multiplies blocking-copy bandwidth by mean per-layer TTFT or TPOT and ignores concurrent kernels.

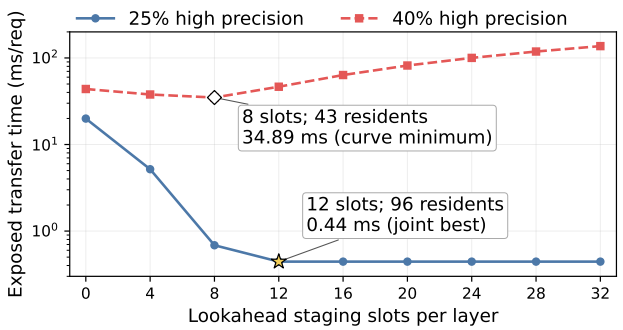


Fig. 4. Trace replay of the two quality- and memory-feasible fidelity allocations on Qwen3-30B, at the measured host-to-device bandwidth of this trace. Increasing the high-precision share from 25% to 40% reduces residency and moves the best staging allocation from 12 to 8 slots. Bandwidth is not swept. Lookahead uses the measured active set. Section 5 is the joint runtime.

Figure 1(b) the same shares sit 0.54 and 0.38 above all-high, $6.280 - 5.740$ and $6.120 - 5.740$, and the 10% share and the all-low point sit 0.62 and 0.66 above it, $6.360 - 5.740$ and $6.400 - 5.740$. ϵ is the smallest one-decimal number that is at least this 25% gap, so $\epsilon = 0.70$ on Qwen3-30B and 0.60 on Qwen3-Next-80B. On Figure 1(c), the point with 75% of experts at low precision is a 25% high-precision share. It sits 0.83 above all-high, $6.840 - 6.010$. The same rule gives $\epsilon = 0.90$. An operator fixes ϵ with this rule before the evaluation prompts are read. A point 0.02 to 0.09 above ϵ is outside Eq. (2). Section 6.4 gives the uncertainty of the downstream scores. Table 6 gives the same budgets inside tighter fixed bounds. The raw 25% gaps are not ϵ. Panel (b)'s 0.54 is the WikiText-2 gap $6.280-5.740$. Table 5's Static-Joint ΔPPL is also 0.54, from the mixed-prompt values $6.28-5.74$. Panel (a)'s 0.62 is the WikiText-2 gap. Table 5's 0.62 is $7.03 - 6.41$ on the mixed prompts, and panel (a) ends at 7.19 rather than the 7.29 all-low value in Table 4. Setting ϵ to either mixed-prompt value would place it on one of those rows. Phi-3.5-MoE's WikiText-2 gap is $6.840 - 6.010 = 0.83$. Table 5's 0.83 is HOBBIT's mixed-prompt ΔPPL. Phi-3.5-MoE's mixed-prompt all-low degradation is $6.98 - 6.01 = 0.97$, above this ceiling.

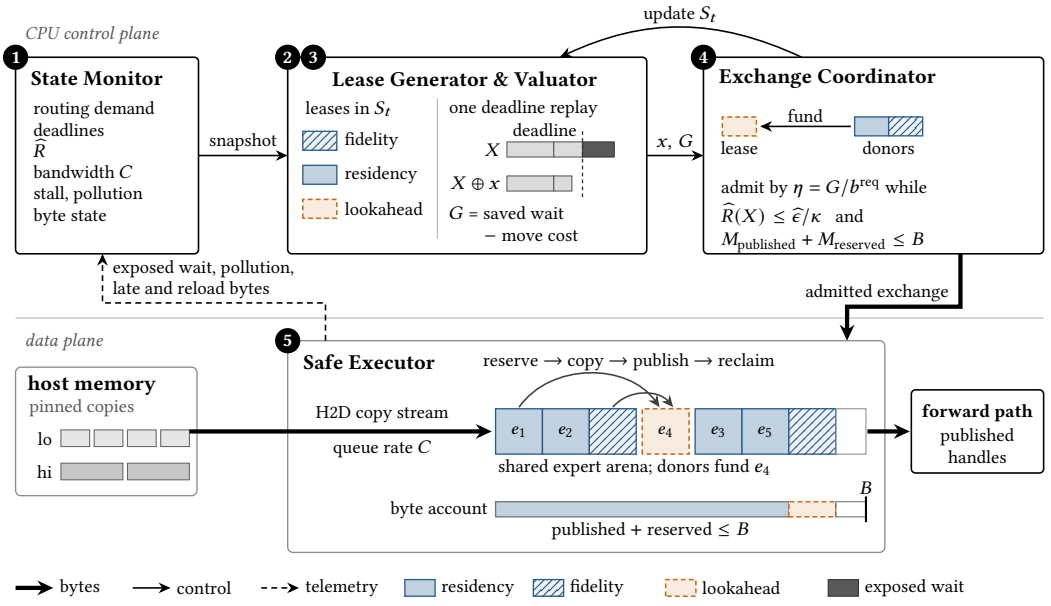


Fig. 5. Admission path. Numbered arrows are the five steps in this section.

2.6 Motivation and Design Challenges

The four measurements are the inputs of Eq. (2). Quantization, caching, and prefetching can already run in one system, and the controller still has to decide which bytes to free.

Which spend is worth the bytes. A high-precision copy, a resident expert, and an early copy are measured in different units and last for different amounts of time. $\widehat{R}$ is not converted into milliseconds or added to the latency objective.

Which decision can wait. Routing and deadlines change faster than the perplexity evidence needed to justify a high-precision copy. The same budget still assigns expert bytes and transfer time, while the evidence and the larger copy take longer to pay for themselves.

Which transition fits. A resident set that fits in B can still pass through a copy that does not. A high-precision copy and an early copy need destination bytes before the transfer starts, the old copy stays occupied until its readers finish, and a copy still in transfer cannot execute.

3 DESIGN

3.1 Overview

DynaByte answers the allocation question with one admission procedure, shown in Figure 5. The procedure chooses each expert's precision, whether it stays on the GPU, and when its copy starts, then commits those choices together: the copies being requested, the experts demoted or evicted to free their bytes, and the transfer schedule.

The State Monitor records routing demand, deadlines, $\widehat{R}$, bandwidth, and which bytes are on the GPU (step 1). The generator proposes high-precision copies, resident experts, and early copies within the next S_t layers (step 2). The coordinator chooses which resident experts to demote or evict to free those bytes, and the valuator replays the copy queue before and after that choice (step 3). The coordinator admits the choice when predicted exposed wait falls and $\widehat{R} \le \widehat{\epsilon}/\kappa$ (step 4).

The executor reserves, copies, and reclaims on that schedule, keeps the byte peak within B, and publishes the new handle only when the side probe measures $\Delta\text{PPL} \le \epsilon$ (step 5). The return path updates the next snapshot and S_t.

The object passed from step 4 to step 5 is

$$x = (Z^+, D, \mathcal{T}), \tag{1}$$

The coordinator scores the GPU state after the demotions, evictions, and new copies have all been applied. The replay charges the shared queue for a demoted or evicted expert's later reload and does not credit one avoided transfer twice. Scoring the new copy's transfer time minus the released expert's transfer time prices the same two copies without that replay.

A policy π assigns every expert a representation, a residency decision, and a transfer issue time. The allocation problem is

$$\min_{\pi} \ \text{P95}(\text{TPOT}; \pi) \quad \text{subject to} \quad M_{\text{peak}}(\pi) \le B, \qquad \Delta Q(\pi) \le \epsilon. \tag{2}$$

Eq. (2) is the offline problem reported in Section 5. The executor enforces $M_{\text{peak}}(\pi) \le B$ on every transition. One control step does not observe the request-level p95. Admission does not read measured ΔPPL. It admits the copies, demotions, and evictions when predicted exposed wait falls and $\widehat{R} \le \widehat{\epsilon}/\kappa$. Section 4 sets $\kappa > 1$ to the maximum of $(\Delta\text{PPL}/\epsilon)/(\widehat{R}/\widehat{\epsilon})$ on the calibration prompts, rounded up to the next hundredth. Algorithm 1 probes the candidate after its extents are reserved inside B and copied on the serving GPU. The previous handles keep serving until that probe measures $\Delta\text{PPL} \le \epsilon$. The state is published only when that reading is within ϵ. If it exceeds ϵ, the last admitted exchange is rolled back and promotions run until the probe is within ϵ or no promotion fits in B. Section 5.5 checks the Qwen3-30B states in Figure 10(b). The deadline replay charges C for the migration-queue time occupied by each exchange.

An expert is absent, resident in the low-precision representation, resident in the high-precision representation, or in transition toward a resident state. Its packed footprints are m_i^{lo} and m_i^{hi}. A resident high-precision representation is the packed low-precision representation plus the fidelity increment δm_i^{fid} in Table 1, and it remains one packed block. Once its copy is published, a lookahead lease becomes residency at the selected tier. The low- and high-precision representations of one absent expert are mutually exclusive. Section 3.5 specifies when a reserved representation may be published and when donor bytes may be reclaimed.

Figure 6 traces one exchange through both budgets. Lookahead e_4 does not fit in the one free unit, and e_1's residency still leaves it short, so the coordinator also releases e_2's fidelity increment. The replay admits the exchange only when e_4's saved wait exceeds the cost of e_1's later reload and e_2's low-precision copy, while the added $\widehat{R}$ remains within $\widehat{\epsilon}/\kappa$.

3.2 State Monitor

The State Monitor performs step 1 in Figure 5: it turns observed runtime facts into one immutable snapshot. The snapshot contains demand and timing, quality, and execution and byte state.

Demand and timing. The monitor records recent routing transitions, per-layer expert-kernel time, and pre-gating logits when the checkpoint exposes them. Section 3.3 converts these observations into future-use probabilities, deadlines, and reuse estimates.

Quality. An offline sensitivity $q_i \ge 0$ is the increase in calibration loss when only expert i uses its low representation. The runtime does not modify q_i. After each forward step, the monitor updates w_i with an exponential average of expert e's router-weighted dispatch frequency. The product $w_i q_i$ estimates how often the current workload exercises that sensitivity. For a planned state X,

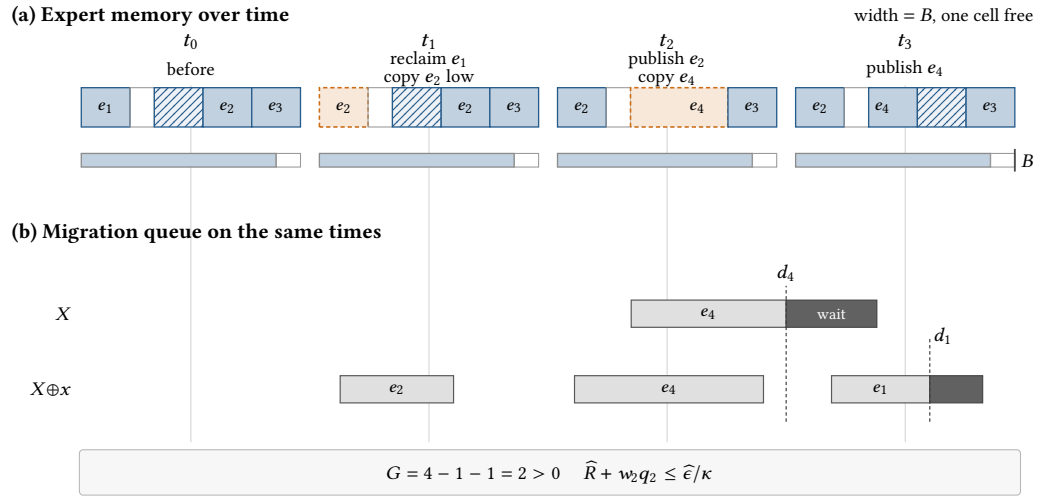


Fig. 6. A closed exchange on one timeline. (a) Every bar has the same width B. The budget is 8 and one cell is free, so published bytes start at 7. e_4 needs 4. Releasing e_1's 2 and e_2's fidelity increment of 2 makes $1+2+2=5$ available. Reserving e_4's 4 restores the free cell to 1, so published plus reserved returns to 7. (b) On that timeline, X misses d_4, and $X \oplus x$ pays for e_2's copy and e_1's reload. The exchange is admitted because $G > 0$ and $\widehat{R} + w_2 q_2 \le \widehat{\epsilon}/\kappa$. The saved wait of e_4 is 4, e_1's reload wait is 1, and $C_{\text{move}} = 1$, so $G = 4 - 1 - 1 = 2 > 0$.

$h_i(X) = 1$ when expert i's next execution uses the high representation. The monitor provides

$$\widehat{R}(X) = \sum_i w_i q_i \big(1 - h_i(X)\big). \tag{3}$$

The coordinator admits against $\widehat{R}(X) \le \widehat{\epsilon}/\kappa$.

Execution and byte state. The monitor separates the published state used by dispatch from the reserved bytes of an accepted transition, and reports their locations and byte counts separately. It also records measured bandwidth, exposed wait, late bytes, unused lookahead bytes, and eviction-induced reload bytes. Stall s_t is the fraction of routed-layer time spent waiting for expert transfers. Pollution p_t is unused lookahead bytes plus reload bytes, divided by the lookahead traffic admitted in that period. The monitor reports the exponentially smoothed values $\widetilde{s_t}$ and $\widetilde{p_t}$. Section 3.4 changes S_t from the prefetch hit rate. Section 3.4 admits the exchange.

The resulting snapshot is the only interface from the monitor to the Lease Generator.

3.3 Which Copies Are Proposed

The generator performs step 2 in Figure 5: it turns the monitor snapshot into the copies proposed for admission. The valuator performs step 3 only after the coordinator has attached the experts to demote or evict, closing that proposal into one exchange $x = (Z^+, D, \mathcal{T})$.

Bound the candidate set. The envelope S_t is the maximum routed-layer depth for candidate generation and allocates no bytes (Table 1). Section 3.4 adjusts S_t from the prefetch hit rate. For a resident set R and predicted expert set $\hat{U}(S)$, the generator retains a prefix depth only when

$$\sum_{i \in \hat{U}(S) \setminus R} m_i \le S T_\ell C. \tag{4}$$

Table 2. How the three uses modify the same byte account, $\widehat{R}$ (Eq. (3)), and migration-queue replay. Fidelity is refreshed every T_{slow} forward steps. Residency is refreshed every T_{fast} routed layers. "Queue effect" is the quantity changed in the replay of Eq. (6).

Use	Bytes	Lifetime	Risk effect	Queue effect
Fidelity	δm_i^{fid}	T_{slow}	removes $w_i q_i$	transfer size
Residency	m_i^{lo}	T_{fast}	none by itself	transfer exists
Lookahead	$m_i^{b_i}$	issue to use	selected tier	issue time

This comparison filters candidates. The subsequent replay still uses individual issue times and deadlines.

Forecast uses and form leases. For each retained layer, the forecaster reads pre-gating logits from the snapshot when they are available and otherwise uses recent routing transitions. It produces a predicted use $j = (i_j, b_j, \omega_j, a_j, d_j)$: expert, representation tier, occurrence probability, earliest issue time, and deadline. The deadline is the latest ready time that introduces no transfer wait.

A lease is the record in Table 1. The interval guides valuation: an early use may shorten a lookahead lease, whereas a late copy retains its bytes until completion or cancellation. Table 2 shows how all three leases enter the byte account, $\widehat{R}$ (Eq. (3)), and migration-queue replay.

A fidelity lease claims δm_i^{fid} beyond the base residency lease and lasts for T_{slow} forward steps. A demotion releases that increment only after the low representation is published. A residency lease renews m_i^{lo} for T_{fast} routed layers; eviction releases it after outstanding readers finish. For an absent expert, the generator emits mutually exclusive low- and high-tier lookahead leases when their resulting $\widehat{R}$ is feasible. A lookahead lease lasts from issue until use, after which it becomes residency at the selected tier or is reclaimed.

Value a closed exchange. The generator sends leases, not scores, to the coordinator. For each lease, the coordinator proposes one or more donor sets D and calls the valuator on the resulting exchange $x = (Z^+, D, \mathcal{T})$. The valuator replays transfers in earliest-deadline-first order. A resident expert is ready at the start of the window. An absent expert's completion time includes bytes already queued and its earliest issue time. Uses of the same expert and tier share one readiness event under continuous batching. The predicted exposed wait is

$$\widehat{L}(X) = \sum_j \omega_j \, [r_j(X) - d_j]_+. \tag{5}$$

$\widehat{L}$ sums probability-weighted exposed wait inside one replay window; Eq. (2) is the request-level p95. Section 5.5 checks both.

The valuator returns

$$G(x \mid X) = \widehat{L}(X) - \widehat{L}(X \oplus x) - C_{\text{move}}(x), \tag{6}$$

where $X \oplus x$ applies the lease, donors, and transition schedule, and C_{move} is exposed transition cost. One replay prevents a retained expert and its lookahead from both receiving credit for the same avoided transfer, and it charges every released donor for its later reload.

3.4 Which Copies Are Admitted

The coordinator decides which experts to release before it makes the only admission decision. One control step has three stages: attach the experts to release, admit by G, and refresh the next candidate set.

Close. The coordinator initializes provisional state X from the monitor snapshot. Free bytes, $B - M_{\text{published}} - M_{\text{reserved}}$, pay for the requested copy when they are sufficient. Otherwise D is built from high-precision copies that can be demoted, resident experts that can be evicted, and early copies that can be cancelled. Copies that have not started, copies that are no longer useful, and residents whose predicted reuse lies beyond S_t are released first. A resident that is still needed is released only when the replay charges its later reload. The scan follows increasing counterfactual loss, the extra predicted wait and move cost caused by the release. In Figure 6, the early copy of e_4 is funded by releasing e_1's residency and the extra high-precision bytes of e_2.

A closed exchange is feasible only when the lease and its donors do not conflict, its resulting $\widehat{R}$ remains within $\widehat{\epsilon}/\kappa$, and its complete transition satisfies

$$M_{\text{published}}(t) + M_{\text{reserved}}(t) \le B \qquad \forall t. \tag{7}$$

A release enters the byte account only at its scheduled reclaim time, so the same bytes cannot pay for two copies, and bytes that become free only later cannot be spent as free bytes now.

If $\widehat{R}$ of the published state exceeds $\widehat{\epsilon}/\kappa$, the coordinator first restores feasibility through promotion exchanges with the smallest replayed latency cost per unit of $\widehat{R}$ removed. This repair uses the same close and valuation steps. Once the state is feasible, a demotion can fund another lease only when the complete resulting exchange remains feasible.

Value and admit. Let b_x^{req} be the requested lease's bytes over its next lifetime, including bytes renewed by a residency lease. The coordinator ranks feasible exchanges with positive G by

$$\eta(x \mid X) = G(x \mid X)/b_x^{\text{req}}. \tag{8}$$

After each admission it updates both provisional accounts, invalidates conflicting requests, and asks the valuator to replay the affected exchanges. Retaining an expert can remove its lookahead lease, and changing a representation can alter several completion times. The search stops when no feasible exchange has positive G. Each control event admits the feasible exchange with the largest $\eta = G/b_x^{\text{req}}$, where G is the drop in predicted exposed wait after the replay of the closed exchange. G depends on exchanges already admitted. η ranks exchanges by the drop in weighted exposed wait inside the current window, per requested byte. Section 5.5 reports when that drop moves window p95. Removing the admissions that raise window p95 is computed after the run and is not the admission rule. Section 5.1 freezes one calibration assignment as Static-Joint. Admission still requires $\widehat{R} \le \widehat{\epsilon}/\kappa$.

Refresh. Every T_{fast} routed layers, the generator refreshes residency and lookahead leases. Every T_{slow} forward steps, it adds fidelity leases to the same set. Minimum fidelity tenure and replacement hysteresis suppress changes too short to repay their copies. Algorithm 1 adjusts S_t, and Section 4 states the Qwen3-30B constants. Refresh changes the candidate set. Admission remains closure, replay, and η.

On a fast event the coordinator submits the surrogate-admitted schedule without a probe. On a slow boundary it submits only when the probe measures $\Delta\text{PPL} \le \epsilon$. Otherwise it leaves the published state unchanged.

3.5 Safe Executor

The Safe Executor performs step 5 in Figure 5. Its input is an admitted schedule $(Z^+, D, \mathcal{T})$. Its output is a new published state and telemetry for the next snapshot. The executor runs the admitted schedule.

Reserve. The executor reserves every extent in $\mathcal{T}$ in one transaction and checks the transition peak in Eq. (7). A destination that must exist before its donor becomes reclaimable is bound to an actual extent during this transaction. If no compatible extent can be bound, or the peak exceeds B, the executor discards the schedule and leaves the published state unchanged.

Copy and publish. Copies run on the migration stream. A demotion keeps serving the high representation until the low replacement is published. An eviction-only donor is reclaimed after its readers finish. The candidate copies reside in the reserved extents and count toward B. Algorithm 1 probes those copies on the serving GPU. The executor publishes the new handle only after that probe measures $\Delta\text{PPL} \leq \epsilon$. Until then the forward path uses the old representation.

Reclaim and ready work. The executor reclaims an old extent only after its reader count reaches zero and its recorded uses complete. After routing, groups with published handles issue before groups waiting on a transfer. A missed lookahead waits only in its assigned group, and routing and top-k do not change. A speculative exchange may displace a protected resident only when the replay of its closed exchange has already charged that resident's reload.

Telemetry. The executor returns the measurements defined in Section 3.2. They update the next snapshot. If that probe measures $\Delta\text{PPL} > \epsilon$, Algorithm 1 rolls back the last admitted exchange. The published handles stay until a later probe is within ϵ.

4 IMPLEMENTATION

We implement DynaByte in Python with PyTorch and HuggingFace Transformers [31, 44]. Three mechanisms below are the ones Section 3 leaves to the implementation: the inputs of the forecaster, the bytes stored for each representation tier, and the placement of one extent in the shared arena.

Forecaster. For each layer retained by S_t, the forecaster in Section 3.3 reads pre-gating logits from the snapshot when the snapshot contains them. Otherwise it ranks that layer's experts by recent activation counts and returns a prefix of configured width. Low confidence lengthens this prefix and leaves S_t unchanged. When the forecast is late for the layer that is about to run, the generator uses recent routing history and the forward path continues. Admission uses this forecast as the predicted use in Section 3.3.

Envelope and representations. At startup S_t is the number of layer intervals required to copy one layer of absent-expert bytes at the copy rate C, clipped to the configured bounds. Section 3.4 then updates that depth from the prefetch hit rate. Before a timed run, both tiers of each layer are packed into pinned host memory, and that layer's source weights are released once both packs exist. The checkpoint fixes the tier pair, so the two representations stay the same as B changes. While a layer runs, each tier that the layer contains is issued as one grouped GEMM.

Shared arena. Each device-resident expert representation occupies one extent, a contiguous byte range that holds that representation. Layer, tier, and lease purpose share this arena. The arena is backed by one or more preallocated segments, each a contiguous buffer, and every extent is taken from a single segment. The allocator rounds a request up to a multiple of 256 bytes and selects the smallest free range, among all segments, that can hold the rounded size. When a range is returned, adjacent free ranges in the same segment are merged.

Operating point. The Qwen3-30B runs instantiate Algorithm 1 with $T_{\text{fast}} = 32$ routed layers, $T_{\text{slow}} = 50$ forward steps, $S_{\min} = 0$, $S_{\max} = 3$, and prefetch-hit thresholds 0.25 and 0.60. The expert-score EMA coefficient is 0.9, minimum fidelity tenure is 100 steps, and the replacement margin is 0.01. Each layer's counterfactual state keeps its 8 highest positive scores. Figure 10(c)

replays K exchange candidates; the marked point is $K = 8$ and $W = 32$. INT4 groups are 128. The Qwen3-Next-80B low tier uses INT2 groups of 64. Sensitivity calibration uses 256 WikiText prompts: 619 s on Qwen3-30B, 2743 s on Qwen3-Next-80B, and 423 s on Phi-3.5-MoE. On these prompts, and with the ϵ values from Section 2.5, the maximum of $(\Delta\text{PPL}/\epsilon)/(\widehat{R}/\widehat{\epsilon})$ is 1.09, 1.07, and 1.05. κ is that maximum rounded up to the next hundredth, so $\kappa = 1.10$, 1.08, and 1.06. These three values are fixed before the evaluation prompts are read. The probe in Algorithm 1 runs only at a slow boundary, and only when the candidate would change the published fidelity set. Fidelity tenure is 100 steps, so a new probe does not start before that tenure ends. It does not run on a fast event. Each probe is 4 sequences of 16 tokens, batched together. One step is charged at that model's Table 5 p95 TPOT, so 16 steps take $16 \times 45.8 = 732.8$ ms, $16 \times 141 = 2256$ ms, and $16 \times 53.8 = 860.8$ ms. The 256-prompt sensitivity pass scores ablated assignments. This probe scores one candidate. These 4 sequences are not among the 256 calibration prompts and are not evaluation prompts. Their all-high log-probabilities are stored before serving. The probe scores the candidate on these sequences, turns that score and the stored score into perplexity, and takes ΔPPL as the candidate perplexity minus the stored all-high perplexity. On the Table 5 streams the fidelity set stays fixed, and the probe finishes before those timed requests. The host is an Intel Xeon Gold 6338 with 1 TB of memory; the GPU is an RTX A6000 (48 GB), on PCIe, with PyTorch 2.10.0+cu128 and Transformers 5.12.1.

5 EVALUATION

Five questions follow: tail latency at fixed expert-memory budgets (RQ1), a shared online assignment against a fixed order and a frozen one (RQ2), which of the three removals in Figure 9 moves the Table 5 gap (RQ3), whether the online surrogates and the controller are cheap enough to run (RQ4), and how offered load changes end-to-end latency and SLO goodput (RQ5).

5.1 Experimental Setup

Platform and models. We evaluate on a single NVIDIA RTX A6000 GPU with 48 GB of device memory. Table 3 summarizes each checkpoint's routing structure and the two packed tiers of routed experts. For Qwen3-Next, the shared expert is not routed and remains resident with the non-expert weights.

For Qwen3-30B, the expert arena occupies 29.74 GB, corresponding to $\Phi = 1.931$. The low- and high-tier expert packs occupy 15.40 GB and 57.98 GB, respectively, giving $\rho = 3.76$; keeping both packs resident would require 73.38 GB and therefore exceeds device capacity. Host-to-device bandwidth is 24.86 GB/s.

Table 5 and the later latency figures for this model use $B = 20.0$ GB ($\Phi = 1.3$), 9.74 GB below the 29.74 GB arena. The $\Phi = 0.7$ run in Section 5.2 uses $B = 10.78$ GB.

Workloads and quality. The prompts are drawn from a mixed workload comprising MMLU-Pro [43], GPQA [35], AIME25 [52], GSM8K [6], and HumanEval [3]. We construct workload slices according to the input/output token-length ratio: prefill-heavy requests have the largest input-to-output ratios, decode-heavy requests have the largest output-to-input ratios, and balanced requests minimize the difference between input and output lengths. Unless otherwise specified, inference uses a batch size of 8. We measure model-path time to first token (TTFT) and time per output token (TPOT) on a fixed replay order. Section 5.6 offers the same prompts from an open-loop client and includes queue wait, serialization, loopback transport, tokenization, offered load, and goodput.

Calibration prompts determine the sensitivity estimates, predictor width, controller constants, κ, and baseline configurations. Evaluation prompts are disjoint from the calibration prompts and from

Table 3. Configuration of the evaluated MoE models. High and low tiers are the packed representations of each routed expert.

	Qwen3-30B	Qwen3-Next-80B	Phi-3.5-MoE
Total parameters	30.5B	80B	42B
Activated / token	3.3B	3B	6.6B
High tier	FP16	INT4	FP16
Low tier	INT4	INT2	INT4
Expert weight fraction	95%	96%	96%
Layers	48	48	32
Routed experts / layer	128	512 (+1 shared)	16
Top-K	8	10	2

the 4 probe sequences, and no parameter is refit on the test set. Static-Joint keeps the assignment frozen from the calibration mix. DynaByte may change the assignment on the evaluation stream.

All-high is the quality reference: every routed expert uses the high tier, and a missing expert is fetched on the residency path used by the other policies.

The scores in Table 4 use 200 MMLU-Pro items, 198 GPQA items, 30 AIME25 items, 200 GSM8K items, and 164 HumanEval items. Avg. weights the five accuracies by these counts.

Baselines. Each baseline fixes one of these decisions. *Uniform-low* places every routed expert at the low tier. *Static-mixed* keeps that low-tier pack resident and spends $B - P_{\text{lo}}$ on fidelity increments for the largest-q_i experts that fit, then freezes the assignment. At the Table 5 budgets $\Phi \geq 1$, lookahead is off, so no expert is transferred on the measured requests. *Residency-only* leaves $B - P_{\text{lo}}$ idle and *Lookahead-only* stages no expert; both are the Uniform-low point. At $B = 10.78$ GB the low-tier pack does not fit, so Static-mixed adds no fidelity increment and absent experts transfer at low precision. Seq-FR and Seq-RF test whether independently optimizing the same decisions in a fixed order is sufficient: Seq-FR assigns fidelity before residency and lookahead, whereas Seq-RF assigns residency before fidelity and lookahead. Static-Joint assigns B among the three uses on calibration statistics and keeps the feasible assignment with the lowest latency. It then freezes that assignment on the evaluation stream. *Fixed-partition* instead reserves independent quotas for the three uses, testing the value of allowing memory to move across them.

For controlled comparisons, every DynaByte policy receives the same expert-memory budget B, expert representations, kernels, request stream, and transition interface. Baselines that require calibration receive the same calibration traces and search budget as DynaByte. HOBBIT [42] and DyMoE [20] have no public implementation. We reimplemented both from the papers under the same expert-memory budget, packed tiers, kernels, and request stream. HOBBIT uses its dynamic loader, adaptive prefetcher, and multi-level cache. In this comparison a cache miss on a less critical expert loads the low-precision copy. DyMoE uses its importance estimator, depth-aware schedule, cache, and lookahead prefetcher. That schedule assigns the high and low tiers already packed for the comparison, FP16/INT4 on Qwen3-30B and Phi-3.5-MoE and INT4/INT2 on Qwen3-Next-80B, and every expert the router selected is executed. DyMoE's published 4/0 schedule omits routed experts. Executing every expert the router selected keeps Table 5's ΔPPL on the same top-k set as the other rows. The 4/0 schedule is not one of those rows. HOBBIT's other measured points at these budgets are its rows in Table 6. On the same kernels and request stream, that 4/0 schedule has ΔPPL 0.81, 0.71, and 0.96, above $\epsilon = 0.70$, 0.60, and 0.90. Its p95 centers are 49.4 ms [48.0, 50.8], 170 ms [161, 179], and 69.5 ms [65.7, 73.3]. Each bracket overlaps the DyMoE bracket in Table 5.

Metrics and statistics. We report time to first token (TTFT), time per output token (TPOT), and exposed expert-transfer wait on the 792 requests counted above. Each run uses five warm-up iterations and then those 792 requests on each of five seeds. Policies that share a seed see the same request order. For each policy and seed, the request-level p95 TPOT is computed on that seed's 792 requests. We report the median of those five p95 values, and the bracket is their minimum and maximum. TTFT is the request-level p95 on the same requests. Comparisons use these centers. Where one bracket lies entirely below another, every paired seed difference has that sign, and the difference is at least the gap between the brackets.

Memory accounting and admission. A run is valid only if arena occupancy remains within the expert-memory budget B and its exchange log can reconstruct the reported allocation. NVML samples process-level GPU memory every 2 ms, and Table 5 reports that high-water mark as Peak.

5.2 Model-Path Quality–Latency Tradeoff

RQ1: Tail latency at fixed expert-memory budgets. The latency in this section is the model-path TTFT and TPOT of Section 5.1, on one GPU at a fixed expert budget. Each model is evaluated at one expert-memory budget. Every measured knob at that budget is plotted. Static-mixed spends only the slack $B - P_{\text{lo}}$ on fidelity and transfers no expert, at ΔPPL 0.79, 0.62, and 0.93, and it is faster by 2.1 ms, 24 ms, and 1.9 ms.

Table 4. Mixed-prompt perplexity and downstream scores. High and Low are the all-high and all-low references.

Model	Tier	Mixed PPL	MMLU-Pro	GPQA	AIME25	GSM8K	HumanEval	Avg.
Qwen3-30B	High	6.41	76.00	54.04	33.33	90.50	84.76	74.37
	Low	7.29	73.50	50.51	30.00	89.50	84.15	72.35
	Static-Joint	7.08	75.00	53.03	33.33	90.00	84.76	73.74
	DynaByte	7.03	74.50	53.03	33.33	89.50	84.76	73.49
Qwen3-Next-80B	High	5.74	76.00	72.22	70.00	87.00	85.37	79.55
	Low	6.40	71.50	65.66	60.00	86.00	84.76	76.01
	Static-Joint	6.28	75.00	71.21	66.67	87.00	85.37	78.91
	DynaByte	6.26	75.50	71.21	66.67	87.00	85.37	79.04
Phi-3.5-MoE	High	6.01	54.00	36.87	40.00	88.50	70.73	61.36
	Low	6.98	52.00	34.34	36.67	87.50	70.12	59.72
	Static-Joint	6.77	53.50	35.86	40.00	88.00	70.73	60.86
	DynaByte	6.83	53.00	35.36	40.00	88.50	70.73	60.73

Table 4 reports perplexity on the mixed evaluation prompts and the five downstream tasks. DynaByte's averages are 73.49, 79.04, and 60.73, and the all-high averages are 74.37, 79.55, and 61.36. The differences from all-high are 0.88, 0.51, and 0.63 points (1.18%, 0.64%, and 1.03%). Those gaps are 7, 4, and 5 items of the 792. Section 6.4 gives the one-item movement and the normal half-width of each column. The differences from all-low are 1.14, 3.03, and 1.01 points. Against Static-Joint, DynaByte is 0.25 and 0.13 points lower on Qwen3-30B and Phi-3.5-MoE, and 0.13 points higher on Qwen3-Next-80B.

Figure 7 shows the corresponding quality–latency frontier at one expert-memory budget per model: Qwen3-30B at $\Phi = 1.3$ ($B = 20.0$ GB), Qwen3-Next-80B at $\Phi = 1.4$ ($B = 28.0$ GB), and Phi-3.5-MoE at $\Phi = 1.2$.

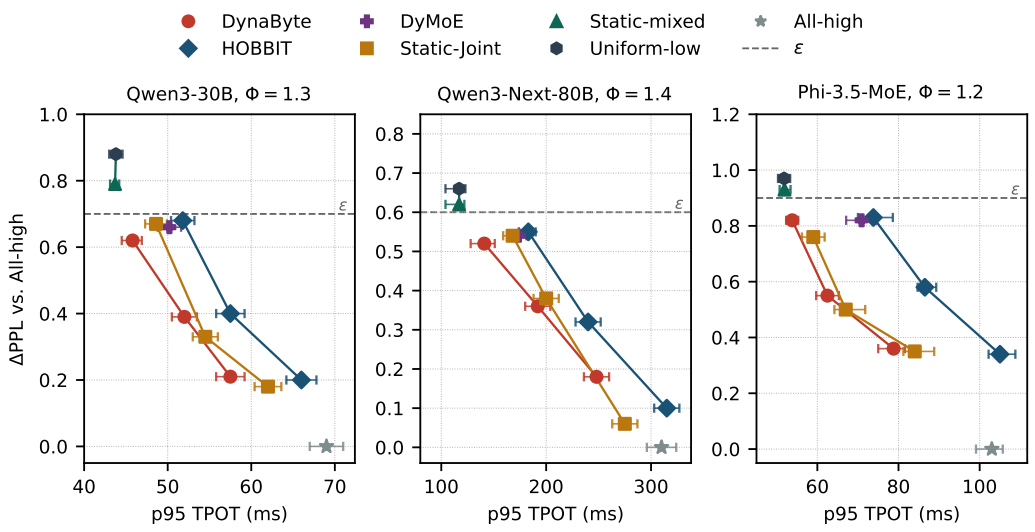


Fig. 7. Quality–latency frontier at one expert-memory budget per model. Horizontal bars are the brackets of Section 5.1. DyMoE's marker is its row in Table 5; the other markers of DynaByte, HOBBIT, and Static-Joint are the knobs in Table 6. Static-mixed and Uniform-low are joined because no expert is transferred between those two assignments.

Table 5 reports the corresponding tail-latency results. Static-mixed's p95 centers are 43.7 ms [43.1, 44.2], 117 ms [104, 122], and 51.9 ms [50.7, 53.4]. Uniform-low's centers are 43.8 ms [43.0, 44.6], 117 ms [104, 123], and 51.8 ms [50.3, 53.3]. Static-mixed's ΔPPL values are 0.79, 0.62, and 0.93. Uniform-low's are 0.88, 0.66, and 0.97. Static-mixed lies 0.09, 0.02, and 0.03 above $\epsilon = 0.70$, 0.60, and 0.90. Uniform-low lies 0.18, 0.06, and 0.07 above them. On Qwen3-30B, $B - P_{\text{lo}} = 4.60$ GB. One fidelity increment is $(57.98 - 15.40)/128$ GB, so that slack holds 13 of 128 experts and leaves 0.28 GB. Recomputing the assignment Static-Joint freezes in Section 5.1 lowers the center by 2.8 ms and 27 ms on Qwen3-30B and Qwen3-Next-80B, where DynaByte is also faster at a lower ΔPPL, 0.62 versus 0.67 and 0.52 versus 0.54. On Phi-3.5-MoE, DynaByte is at ΔPPL 0.82 and 53.8 ms, and Static-Joint is at 0.76 and 59.0 ms. Both ΔPPL values are within $\epsilon = 0.90$. Static-Joint's ΔPPL is 0.06 lower and its average in Table 4 is 0.13 higher, and the latency gap is 5.2 ms. Against Static-Joint, the paired gap is at least $47.3 - 46.9 = 0.4$ ms, $159 - 151 = 8$ ms, and $56.2 - 55.3 = 0.9$ ms. HOBBIT's p95 center is above DynaByte's on every model: 51.8 ms and 45.8 ms, 183 ms and 141 ms, and 73.8 ms and 53.8 ms. DyMoE measures ΔPPL 0.66, 0.54, and 0.82. All three are within $\epsilon = 0.70$, 0.60, and 0.90. Its p95 centers are 50.2 ms [48.8, 51.6], 174 ms [165, 183], and 70.9 ms [67.1, 74.7]. Each center is above DynaByte, and those three intervals do not overlap DynaByte's. Against HOBBIT, the paired gap is at least $50.4 - 46.9 = 3.5$ ms, $174 - 151 = 23$ ms, and $70.0 - 55.3 = 14.7$ ms. Against DyMoE, it is at least $48.8 - 46.9 = 1.9$ ms, $165 - 151 = 14$ ms, and $67.1 - 55.3 = 11.8$ ms. Each center is below HOBBIT, and all three intervals overlap HOBBIT's. One point dominates another when both its ΔPPL and its p95 are lower. On Qwen3-30B, 0.62 at 45.8 ms has lower ΔPPL and lower p95 than HOBBIT at 0.68 and 51.8 ms, DyMoE at 0.66 and 50.2 ms, and Static-Joint at 0.67 and 48.6 ms. It does not dominate Static-mixed at 0.79 and 43.7 ms or Uniform-low at 0.88 and 43.8 ms. The Qwen3-30B knob 0.39 at 52.0 ms dominates HOBBIT at 0.40 and 57.5 ms and does not dominate Static-Joint at 0.33 and 54.5 ms. On Qwen3-Next-80B, 0.52 at 141 ms dominates HOBBIT at 0.55

Table 5. Loose setting of each policy in Figure 7. No row is removed for exceeding its bound. Brackets apply only to p95 TPOT (Section 5.1). Peak includes non-expert weights, the KV cache, and activations, and is not the expert-memory budget B.

Model	Policy	ΔPPL	p95 TPOT (ms)	p95 TTFT (s)	Peak (GB)
Qwen3-30B	DynaByte	0.62	45.8 [44.5, 46.9]	3.1	38.6
	HOBBIT	0.68	51.8 [50.4, 53.2]	3.8	38.6
	DyMoE	0.66	50.2 [48.8, 51.6]	3.6	38.6
	Static-Joint	0.67	48.6 [47.3, 49.9]	3.2	38.6
	Static-mixed	0.79	43.7 [43.1, 44.2]	2.6	38.6
	Uniform-low	0.88	43.8 [43.0, 44.6]	2.6	34.0
Qwen3-Next-80B	DynaByte	0.52	141 [128, 151]	8.2	43.8
	HOBBIT	0.55	183 [174, 190]	10.4	43.8
	DyMoE	0.54	174 [165, 183]	9.9	43.8
	Static-Joint	0.54	168 [159, 181]	8.6	43.8
	Static-mixed	0.62	117 [104, 122]	6.8	43.8
	Uniform-low	0.66	117 [104, 123]	6.8	35.8
Phi-3.5-MoE	DynaByte	0.82	53.8 [52.3, 55.3]	2.2	47.1
	HOBBIT	0.83	73.8 [70.0, 78.6]	2.9	47.1
	DyMoE	0.82	70.9 [67.1, 74.7]	2.8	47.1
	Static-Joint	0.76	59.0 [56.2, 61.8]	2.3	47.1
	Static-mixed	0.93	51.9 [50.7, 53.4]	1.8	47.1
	Uniform-low	0.97	51.8 [50.3, 53.3]	1.8	43.8

Table 6. Other measured knobs at the same budgets as Table 5. These rows are not selected by ϵ. Brackets apply only to p95 TPOT.

	Qwen3-30B		Qwen3-Next-80B		Phi-3.5-MoE	
Policy	ΔPPL	p95 TPOT (ms)	ΔPPL	p95 TPOT (ms)	ΔPPL	p95 TPOT (ms)
DynaByte	0.39	52.0 [50.5, 53.5]	0.36	192 [180, 204]	0.55	62.5 [59.7, 65.3]
DynaByte	0.21	57.5 [55.8, 59.2]	0.18	248 [236, 260]	0.36	78.8 [75.0, 81.6]
HOBBIT	0.40	57.5 [55.8, 59.2]	0.32	240 [228, 252]	0.58	86.5 [84.7, 89.3]
HOBBIT	0.20	66.0 [64.2, 67.8]	0.10	315 [303, 327]	0.34	105.0 [102.2, 108.8]
Static-Joint	0.33	54.5 [53.0, 56.0]	0.38	200 [188, 212]	0.50	67.0 [64.2, 71.8]
Static-Joint	0.18	62.0 [60.4, 63.6]	0.06	275 [263, 287]	0.35	84.0 [81.2, 88.8]
All-high	0.00	69.0 [67.0, 71.0]	0.00	310 [296, 324]	0.00	103 [99.1, 105.7]

and 183 ms, DyMoE at 0.54 and 174 ms, and Static-Joint at 0.54 and 168 ms. It does not dominate 0.62 at 117 ms or 0.66 at 117 ms. The knob 0.36 at 192 ms dominates Static-Joint at 0.38 and 200 ms and does not dominate HOBBIT at 0.32 and 240 ms. On Phi-3.5-MoE, DynaByte, HOBBIT, DyMoE, and Static-Joint are within $\epsilon = 0.90$. Static-mixed and Uniform-low are above it. Among the points with $\Delta\text{PPL} \leq 0.90$, 53.8 ms is the lowest p95. The point 0.82 at 53.8 ms dominates HOBBIT at 0.83 and 73.8 ms and, at the same 0.82, is faster than DyMoE at 70.9 ms. It does not dominate Static-Joint at 0.76 and 59.0 ms. It has lower ΔPPL and higher p95 than Static-mixed at 0.93 and 51.9 ms and Uniform-low at 0.97 and 51.8 ms. No DynaByte knob dominates a Static-Joint knob on this model. The knob at 0.55 and 62.5 ms is also within $\epsilon = 0.90$. It is slower than 53.8 ms. It dominates HOBBIT at 0.58 and 86.5 ms and does not dominate Static-Joint at 0.50 and 67.0 ms.

Table 6 lists the other knobs at these budgets. The first DynaByte row is inside 0.40, 0.40, and 0.60: 0.39 at 52.0 ms, 0.36 at 192 ms, and 0.55 at 62.5 ms. The second DynaByte row is inside 0.30, 0.20, and 0.40: 0.21 at 57.5 ms, 0.18 at 248 ms, and 0.36 at 78.8 ms.

At $\Phi = 0.7$ ($B = 10.78$ GB), 15.40 GB of low-tier experts do not fit. Every policy has $\Delta\text{PPL} = 0.88$, the all-low degradation $7.29 - 6.41$ in Table 4, above $\epsilon = 0.70$. The same ΔPPL means this is not a quality comparison; Section 6.1 gives the promotion step. These runs are not points of Figure 7. Every policy fills B, and the process high-water mark is 29.4 GB. The p95 TPOT centers are 47.0 ms [45.8, 48.2] for DynaByte, 53.6 ms [52.2, 55.0] for HOBBIT, 52.2 ms [50.9, 53.5] for DyMoE, 50.9 ms [49.6, 52.2] for Static-Joint, 47.9 ms [47.1, 48.7] for Static-mixed, and 48.4 ms [47.6, 49.2] for Uniform-low. DynaByte's bracket does not overlap HOBBIT, DyMoE, or Static-Joint. It overlaps Static-mixed and Uniform-low, so those center gaps are not resolved. The p95 TTFT values, in that same order, are 3.3 s, 4.1 s, 3.8 s, 3.4 s, 2.9 s, and 3.0 s.

5.3 Shared Online Allocation and Simpler Policies

RQ2: When does a shared online allocation improve on a simpler policy? Seq-FR commits bytes to fidelity first, and Seq-RF commits them to residency first. On the stationary balanced-length slice neither assignment can move those bytes to the other use (Figure 8(a)).

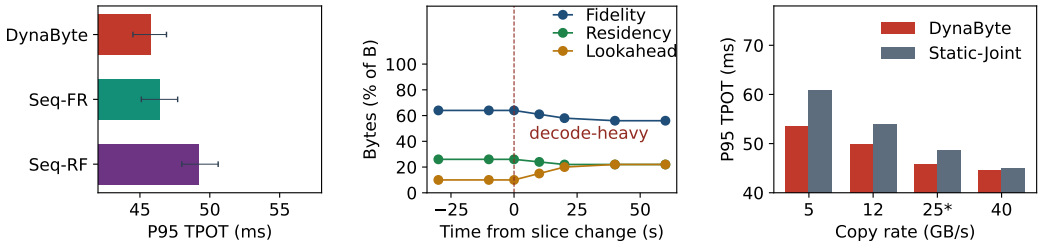
After the slice becomes decode-heavy, lookahead's share rises and fidelity's falls (Figure 8(b)). Decode leaves less time before use. On this Qwen3-30B slice, two fidelity publications finish before that measured interval. Each probe is the 16-step probe in Section 4, $16 \times 45.8 = 732.8$ ms, so the two probes take 1465.6 ms. Each publication then copies one expert, $(57.98 - 15.40)/128 = 0.3327$ GB at 24.86 GB/s, which is 13.4 ms; the two copies take 26.8 ms. Both probes are within ϵ, so the rollback time is 0 ms. A request that overlaps one publication waits $732.8 + 13.4 = 746.2$ ms for the serving GPU. Table 5's p95 remains the service time of its timed requests, and the p95 in Figure 8(b) remains the service time of its measured interval.

Copy rate. The gap to Static-Joint is 7.2 ms, 4.2 ms, 2.8 ms, and 0.4 ms at 5, 12, 25, and 40 GB/s. It shrinks as the copy fits inside the overlap window. The gap is 2.6 ms, 2.6 ms, and 2.8 ms at batches 1, 2, and 4, equals the Table 5 gap of 2.8 ms at batch 8, and widens to 6.2 ms and 7.4 ms at batches 16 and 32.

5.4 Mechanism Attribution

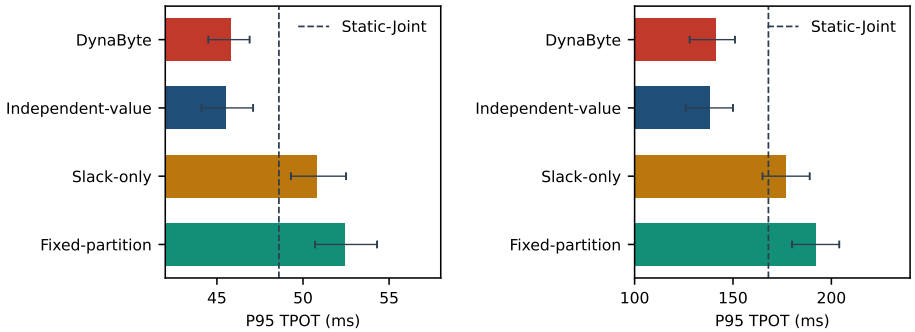
RQ3: Which of these removals moves the Table 5 gap? We isolate the contribution of DynaByte's key mechanisms by removing one at a time while holding the workload, expert-memory budget B, and execution path fixed within each experiment. *Independent-value* keeps the shared arena, the $\widehat{R}$ cap, and the executor. It scores the requested lease and its donor as separate transfer times, $p_{\text{req}}m/C - p_{\text{donor}}m/C$, and does not replay the shared queue. *Slack-only* restricts admission to currently available slack and disables donor selection. *Fixed-partition*, defined in Section 5.1, replaces the shared arena with separate quotas for fidelity, residency, and lookahead.

Donor selection and the shared arena move the Table 5 gaps. Slack-only's centers are 50.8 ms and 177 ms, 5.0 ms and 36 ms above DynaByte, because an exchange that does not fit in slack has no donor. Fixed-partition's centers are 52.4 ms and 192 ms, above DynaByte and above the Static-Joint centers 48.6 ms and 168 ms, because a separate quota leaves bytes in the use that does not need them. Independent-value is that separate benefit and cost. Its centers are 45.5 ms [44.1, 47.1] and 138 ms [126, 150], 0.3 ms and 3 ms below DynaByte, and both intervals overlap DynaByte's [44.5, 46.9] and [128, 151]. The joint replay's extra term is not resolved on this stream. Slack-only and Fixed-partition are the removals that move the gap to Static-Joint.



(a) Table 5 reference (b) Balanced to decode-heavy (c) Host-to-device copy rate

Fig. 8. When shared and online allocation is useful for Qwen3-30B at $\Phi = 1.3$. (a) DynaByte's bar is the mixed-stream row of Table 5, drawn beside Seq-FR and Seq-RF. This bar is not a run on the balanced-length slice. Seq-FR and Seq-RF have no ΔPPL on the mixed-prompt scale, so none is printed. (b) At $t = 0$ the slice changes and the shares are still 64/26/10. They then move to 56/22/22, which is reached at $t = 40$ s. When the fidelity share changes, the probe in Algorithm 1 finishes before the measured interval. The p95 in this panel is the service time of that interval. With $\rho = 3.76$, a fidelity share f keeps a residency share of at least $f/2.76$. (c) The marked point is the Table 5 configuration at the native bandwidth and batch size 8, so it uses that row's brackets.



(a) Qwen3-30B, FP16/INT4 (b) Qwen3-Next-80B, INT4/INT2

Fig. 9. Ablation of DynaByte's allocation mechanisms. (a) Qwen3-30B at $\Phi = 1.3$. (b) Qwen3-Next-80B at $\Phi = 1.4$. Brackets: Section 5.1. The dashed line is the Table 5 Static-Joint center.

5.5 Surrogate Checks and Runtime Cost

RQ4: Are the online surrogates accurate and inexpensive enough for runtime decisions? Figure 10 checks the level of predicted exposed wait, $\widehat{R}$, and controller time for Qwen3-30B. Lookahead recall at depth 3, the fraction of the later demanded expert set found in time to affect allocation, is 0.71 on prefill-heavy requests, 0.58 on mixed requests, and 0.46 on decode-heavy requests.

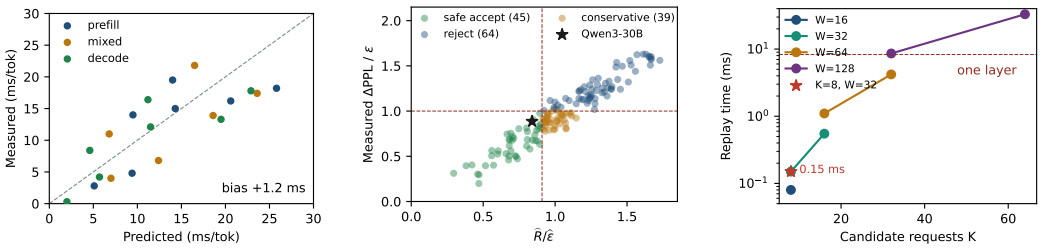
Figure 10(a) compares the level of $\widehat{L}$ with measured exposed transfer wait. On this workload the mean bias is +1.2 ms per token. Individual points lie on both sides of the diagonal. The coordinator admits on $G = \widehat{L}(X) - \widehat{L}(X \oplus x) - C_{\text{move}}(x)$ (Eq. (6)). Admission ranks by η (Eq. (8)).

Figure 10(b) plots 148 states for Qwen3-30B. The horizontal axis is $\widehat{R}/\widehat{\epsilon}$, and the vertical axis is measured $\Delta\text{PPL}/\epsilon$. Admission is the vertical line at $1/\kappa = 1/1.10$. A safe accept lies on or left of that line and on or below the horizontal line at 1. No state in that region has measured ΔPPL above ϵ. A conservative reject lies to the right of the vertical line and on or below 1. A reject lies

to the right of the vertical line and above 1. The counts are 45 safe accepts, 0 unsafe accepts, 39 conservative rejects, and 64 rejects. Five of the rejects have measured ΔPPL of median 1.04ϵ and maximum 1.12ϵ.

Figure 10(c) measures deadline replay as the number of exchange candidates K and the replay window W increase. At $K = 8$ and $W = 32$, that replay is 0.15 ms. K counts exchange candidates, not the per-layer score cap. One fast event runs every $T_{\text{fast}} = 32$ routed layers, and fidelity leases join it on the $T_{\text{slow}} = 50$ boundary. The largest plotted replay is 33 ms, at $K = 64$ and $W = 128$. The other plotted points lie between 0.08 ms and 8.6 ms.

The Qwen3-30B and Qwen3-Next-80B rows of Table 5 have $\widehat{R}/\widehat{\epsilon} = 0.84$ and 0.82, below $1/1.10$ and $1/1.08$. Their measured $\Delta\text{PPL}/\epsilon$ values are $0.62/0.70 = 0.886$ and $0.52/0.60 = 0.867$. The Phi-3.5-MoE loose row has $\widehat{R}/\widehat{\epsilon} = 0.90$, below $1/1.06$, and measured $\Delta\text{PPL} = 0.82$. Against $\epsilon = 0.90$, $\Delta\text{PPL}/\epsilon = 0.82/0.90 = 0.911$, and $(0.82/0.90)/0.90 = 1.01$, below the calibration maximum 1.05. The star in Figure 10(b) marks the Qwen3-30B row. The other two rows are not on that panel. The same regions, with the vertical line at $1/1.08$ for Qwen3-Next-80B and at $1/1.06$ for Phi-3.5-MoE, give 38 safe accepts, 0 unsafe accepts, 44 conservative rejects, and 66 rejects on 148 states, and 31, 0, 47, and 70 on 148 states. Those states are not drawn on the panel. By that definition, the Qwen3-Next-80B and Phi-3.5-MoE rows of Table 5 are safe accepts. Among admitted states the unsafe counts are 0 of 45, 0 of 38, and 0 of 31. For zero counts the 95% upper bound is $1 - 0.05^{1/n}$, which is 0.064, 0.076, and 0.092. These bounds are the uncertainty of this count. Publication checks the 4 probe sequences, and the workload measurement is the Table 5 ΔPPL. This count is not a guarantee for other workloads. Among states with measured $\Delta\text{PPL} \le \epsilon$, the fraction not admitted is $39/84 = 0.46$, $44/82 = 0.54$, and $47/78 = 0.60$, each with normal half-width $1.96\sqrt{\hat{p}(1-\hat{p})/n} = 0.11$. The same side probe decides publication for all three models. On the evaluation streams it accepts the published states, the DynaByte rows of Table 5. κ is fixed on the 256 WikiText prompts in Section 4. Figure 2 shows different hot experts for WikiText, GSM8K, and HumanEval at layer 15 of Qwen3-30B. The ΔPPL of the three Table 5 rows is measured on the five-task evaluation prompts.



(a) Wait level (b) $\widehat{R}$ (c) Replay cost

Fig. 10. Validation of the online decision signals and controller cost for Qwen3-30B. In (a) the diagonal denotes equal levels.

On Qwen3-30B, the Table 5 centers sit 2.0 ms, 8.0 ms, and 4.8 ms above Uniform-low's 43.8 ms for DynaByte, HOBBIT, and Static-Joint. The 2.8 ms gap is the difference of those first and third remainders. The same subtraction from the 117 ms and 51.8 ms Uniform-low centers applies on Qwen3-Next-80B and Phi-3.5-MoE. On Qwen3-30B the intervals [44.5, 46.9] and [43.0, 44.6] overlap, so the 2.0 ms center gap is not resolved. On Qwen3-Next-80B, [128, 151] and [104, 123] do not overlap. On Phi-3.5-MoE, [52.3, 55.3] and [50.3, 53.3] overlap. These remainders are differences of p95 centers. Table 7 audits G on five control events at $K = 8$, $W = 32$, the star in Figure 10(c). The sign of G matches the measured exposed-wait change on 36 of 40 exchanges. Within an event, the

median Spearman correlation of η with that change is 0.93. Four of 25 admitted exchanges raise window p95. Rejecting them moves the full-run center from 45.8 ms to 45.4 ms, inside [44.5, 46.9]. One admitted exchange lowers mean exposed wait and raises window p95. Of the exchanges whose measured exposed wait falls, 22 of 23 also lower window p95. Those 22 exchanges are the cases where the window sum and the window quantile move together. The other exchange lowers the sum and raises the quantile, so the wait it removes sits below the quantile. Admission does not read window p95. The rejection above is computed after the run and is not the admission rule.

Table 7. Decision audit of G for Qwen3-30B at $K = 8$, $W = 32$, the star in Figure 10(c). Rejecting the harmful admissions is one counterfactual on the same stream.

Quantity	Value
Sign agreement	36/40
Median Spearman of η	0.93
Harmful admissions	4/25
Full-run p95 if those are rejected (ms)	45.4
Mean falls and window p95 rises	1/25

5.6 Offered Load

RQ5: End-to-end latency and goodput. On the same GPU and the same expert-memory budgets, a client on this host offers the mixed prompts under Poisson arrivals and under a two-state process that alternates a slower and a faster Poisson rate at the same mean. End-to-end time starts at the client send and includes serialization, loopback transport, tokenization, waiting until the GPU takes the request, and the model path. The GPU runs at most eight requests at once. A completion counts toward goodput when its end-to-end TTFT is at most twice Uniform-low's p95 TTFT in Table 5: 5.2 s, 13.6 s, and 3.6 s. In Table 8, DynaByte's end-to-end bracket meets that objective and HOBBIT's does not. The bracket is the minimum and maximum over five seeds.

Table 8. Open-loop end-to-end TTFT and SLO goodput. The objective is twice Uniform-low's p95 TTFT in Table 5. Brackets are the minimum and maximum over five seeds.

Policy	Qwen3-30B		Qwen3-Next-80B		Phi-3.5-MoE	
	p95 e2e TTFT (s)	Goodput (req/s)	p95 e2e TTFT (s)	Goodput (req/s)	p95 e2e TTFT (s)	Goodput (req/s)
DynaByte	4.8 [4.6, 4.9]	0.82 [0.82, 0.83]	11.0 [11.0, 11.9]	0.27 [0.26, 0.27]	2.9 [2.8, 3.1]	0.77 [0.77, 0.78]
HOBBIT	7.6 [7.3, 7.9]	0.69 [0.67, 0.70]	23.0 [21.4, 23.9]	0.21 [0.21, 0.22]	7.0 [6.6, 7.4]	0.59 [0.56, 0.60]
Static-Joint	5.3 [5.2, 5.6]	0.80 [0.79, 0.81]	14.7 [14.0, 15.6]	0.26 [0.25, 0.26]	3.5 [3.3, 3.8]	0.76 [0.75, 0.77]
Uniform-low	3.4 [3.3, 3.6]	0.85 [0.84, 0.85]	6.8 [6.8, 7.0]	0.27 [0.25, 0.28]	1.9 [1.9, 2.5]	0.79 [0.78, 0.79]

At 0.45, 0.15, and 0.42 requests/s, p95 end-to-end TTFT stays within 0.1 s of that policy's Table 5 p95 TTFT. Serialization, loopback transport, tokenization, and any smaller wait sit inside that 0.1 s. At the rates in Table 8, DynaByte's end-to-end bracket lies below HOBBIT's on every model, and below Static-Joint's on every model. On Qwen3-30B the centers are 4.8 s [4.6, 4.9] and 7.6 s [7.3, 7.9], against Table 5 TTFT of 3.1 s and 3.8 s. The excess over those TTFT values is 1.7 s and 3.8 s, which is the wait before the model path, and the end-to-end gap is wider than the 11.6% TPOT gap for that reason. HOBBIT's excess is likewise larger on Qwen3-Next-80B and Phi-3.5-MoE. The goodput brackets in Table 8 do not overlap for DynaByte and HOBBIT. The gap between these

brackets is at least $7.3 - 4.9 = 2.4$ s, $21.4 - 11.9 = 9.5$ s, and $6.6 - 3.1 = 3.5$ s for end-to-end TTFT, and at least $0.82 - 0.70 = 0.12$, $0.26 - 0.22 = 0.04$, and $0.77 - 0.60 = 0.17$ requests/s for goodput. Under the two-state arrivals, Qwen3-30B goodput is 0.64 requests/s [0.59, 0.65] for DynaByte and 0.40 [0.35, 0.43] for HOBBIT. Uniform-low reaches 0.85 [0.84, 0.85], 0.27 [0.25, 0.28], and 0.79 [0.78, 0.79] requests/s. On Qwen3-Next-80B that bracket overlaps DynaByte's [0.26, 0.27]. Uniform-low's ΔPPL in Table 5 is 0.88, 0.66, and 0.97, above $\epsilon = 0.70$, 0.60, and 0.90. DyMoE and Static-mixed remain in Table 5: Static-mixed's TTFT matches Uniform-low, and DyMoE's TTFT lies between Static-Joint and HOBBIT.

6 DISCUSSION

6.1 Applicability and Boundary Regimes

The latency comparison uses budgets where the low-precision pack fits and the high-precision pack does not, $1 \le \Phi < \rho$. Above ρ, every expert fits at high precision. At 25 GB/s, DynaByte and Static-Joint center at 45.0 ms and 45.6 ms when $\Phi = 1.0$, at 45.8 ms and 48.6 ms when $\Phi = 1.3$, and at 44.4 ms and 44.8 ms when $\Phi = 1.6$. Three measured settings bound this comparison. At the Qwen3-30B arena, $\Phi = 1.931$, the low-tier pack fits, so a resident low-tier set copies no expert. DynaByte and Static-Joint both center at 43.8 ms. Figure 8(c) holds $\epsilon = 0.70$ fixed and varies host-to-device bandwidth. The gap to Static-Joint is 2.8 ms at the measured copy rate and 0.4 ms at 40 GB/s. Below 1, the low-tier pack does not fit. Section 5.2 measures $\Phi = 0.7$ and $B = 10.78$ GB, below the 15.40 GB low-tier pack. Every policy has ΔPPL = 0.88, above $\epsilon = 0.70$. Algorithm 1 admits a promotion only when that promotion satisfies Eq. (7) and brings $\widehat{R}$ to $\widehat{\epsilon}/\kappa$. On this budget every executed row stays at the all-low value, so the repair leaves the run above $\epsilon = 0.70$. The measured ΔPPL on this budget is 0.88, and no promotion fits in $B = 10.78$ GB. At this common ΔPPL, DynaByte's p95 bracket lies below HOBBIT, DyMoE, and Static-Joint, and it overlaps Static-mixed and Uniform-low.

6.2 Guarantees and Estimation Limits

The executor holds the expert-memory peak within B and does not change routing. The controller admits a copy when $\widehat{R} \le \widehat{\epsilon}/\kappa$. A state is published only when the side probe measures ΔPPL $\le \epsilon$. Section 4 fixes κ at 1.10, 1.08, and 1.06 before the evaluation prompts are read. Eq. (3) sums $w_i q_i$ over low-precision experts. Interactions among experts, and a shift in routing, can make that sum differ from measured ΔPPL.

On Qwen3-30B, every state in Figure 10(b) with $\widehat{R} \le \widehat{\epsilon}/1.10$ has measured ΔPPL $\le \epsilon$. The five states at median 1.04ϵ and maximum 1.12ϵ lie to the right of $1/1.10$ and are not admitted. If the probe exceeds ϵ, Algorithm 1 rolls back the last admitted exchange. The published handles stay until a later probe is within ϵ. On the evaluation streams the probe accepts the published states, the DynaByte rows of Table 5. Figure 8(b) records the Qwen3-30B share and p95 change after the slice moves to decode-heavy. It does not plot ΔPPL. A copy staged from the current forecast prices a reload only inside that forecast's layers, so reuse beyond those layers can still be missed. Section 5.5 measures the wait bias, Table 7 is the sign agreement of G, and its rejection row is computed after the run and is not the admission rule. Table 5 is the request-level p95 of Eq. (2).

6.3 System Scope and Deployment Assumptions

The current runtime manages one accelerator and uses host memory as its backing store. Host memory retains both packed representations for the process lifetime; a deployment that cannot do so would need to quantize on demand or read a representation from storage, adding a transition not modeled here. Resident-first execution assumes that expert groups within a layer can be issued

independently. A kernel that issues experts in a fixed order needs a different latency estimate. Expert-parallel execution and an SSD tier are outside this design. They add per-device capacity, a network or storage queue, and load imbalance, so the replay and the byte account have to include those queues. Section 5.6 includes serialization, loopback transport, tokenization, and queue wait on this host. A second GPU, a NIC between machines, and an SSD remain outside this runtime.

6.4 Validity, Reproducibility, and Ethics

Sections 2.2–2.5 motivate the constraints: quality sensitivity, routing variation, and transfer pressure. Their absolute values are not results of the joint runtime; Section 5 is. The measured streams are the five-task mix and the length-slice change in Figure 8(b). The slices do not cover long contexts or multimodal requests, either of which can change routing breadth, the KV reserve, and available overlap. Table 4 weights tasks by item count, so AIME25 is 30 of 792 items. One item moves MMLU-Pro, GPQA, AIME25, GSM8K, and HumanEval by 0.50, 0.51, 3.33, 0.50, and 0.61 points, and moves the average by 0.13 points. For a column, the normal half-width $1.96\sqrt{\hat{p}(1-\hat{p})/n}$ is at most 6.9, 7.0, 17.5, 4.8, and 7.0 points on MMLU-Pro, GPQA, AIME25, GSM8K, and HumanEval. The normal approximation treats items as independent Bernoulli trials. The policy takes measured bandwidth and kernel time as inputs. Kernels, transfer paths, and quantization support can differ across devices.

A result is reproducible under Eq. (2) only when the logs suffice to verify B, ϵ, κ, and the executed state transitions. The study uses public latency traces and quality benchmarks, recruits no participants, collects no new user data, and sends no prompts to a third-party service. Released artifacts will include the HOBBIT and DyMoE runners used for Table 5, under the assumptions in Section 5.1, and will identify source prompts without redistributing material whose license forbids it.

7 RELATED WORK

Prior systems choose a precision, choose which experts stay on the GPU, or choose when a copy starts, often while holding the other two fixed. We group them by that choice, then compare DynaByte directly with systems that already combine the mechanisms.

Which precision is stored. LLM.int8() [9], GPTQ [17], SqueezeLLM [25], AWQ [30], OmniQuant [36], and SmoothQuant [45] fix representations before serving. AQLM [13] and ZeroQuant [49] are further offline compressors for transformer weights. QMoE compresses MoE weights below one bit per parameter [16]. MoE-specific methods use expert sensitivity, activation frequency, or router statistics to assign precision under a memory or quality target [4, 5, 26]. Such methods improve quality for a chosen footprint but commonly evaluate the assignment while which experts stay on the GPU and when copies start stay fixed. DynaByte uses their packed representations and offline sensitivities, and decides at runtime whether a larger high-precision copy is worth the resident experts or early copies it displaces.

Which experts stay on the GPU.. SwapMoE [27], MoE-Infinity [46], EdgeMoE [50], and AdapMoE [54] decide which experts stay and which are evicted from routing history, layer position, or co-activation, and the expert's precision stays fixed.

MoE-Lightning [2], SiDA [11], and Fiddler [24] compute experts that are absent from GPU memory on the CPU when GPU memory is short. PowerInfer keeps frequently activated neurons of sparse feed-forward layers on the GPU and evaluates the remaining neurons on the CPU [41].

Yao et al. place co-activated experts by inter-layer affinity; that communication objective is outside this single-accelerator implementation [48]. DeepSeekMoE specializes experts in the model,

and it is not a placement policy [7]. Tutel [21], DeepSpeed-MoE [33], and MegaBlocks [18] run expert-parallel training and inference across devices. In DynaByte, keeping an expert on the GPU is scored together with the high-precision copy or early copy that would take its bytes, so the cost of evicting it is part of the same decision.

When the copy starts. Cross-layer prefetchers predict future experts and overlap their copies with current computation, using either a fixed depth or a depth adapted from observed demand [14, 22, 40]. Shen et al. schedule that movement for a given expert size [38], and He et al., under the same name, combine a predicted cache with token scheduling [19].

HOBBIT combines dynamic mixed-precision loading, adaptive prefetching, and a multi-level cache; its low-precision miss path reduces transfer cost for experts judged less critical [42]. DyMoE combines online expert importance, depth-aware mixed precision, caching, and lookahead prefetching, and some configurations omit selected experts [20]. HOBBIT and DyMoE already quantize, cache, and prefetch in one system. DynaByte admits a new copy only together with the experts demoted or evicted to free its bytes, and that decision includes the released expert's later reload. Table 5 reports both reimplementations at the same GPU capacity. In that table a HOBBIT miss loads the low-precision copy, and DyMoE keeps every expert the router selected, placing it in one of the two packed tiers. DyMoE's published schedule that drops experts is outside the table. Slack-only and Fixed-partition in Section 5.4 drop donor selection and the shared arena, and those removals move the gap to Static-Joint. Table 5 is the model-path gap to the two reimplementations.

Serving and memory management. vLLM [28] and SGLang [53] manage KV-cache memory, and Orca [51] schedules requests at iteration granularity. Sarathi-Serve [1] chunks long prefills so decode iterations continue. DistServe [55] and Splitwise [32] place prefill and decode on separate resources. ZeRO-Infinity offloads training state across GPU, CPU, and NVMe, and FlexGen schedules dense-model inference across GPU, CPU, and disk [34, 39]. DynaByte allocates the expert-memory budget left after those systems reserve non-expert state. Table 5 reports that budget and the NVML peak separately.

8 CONCLUSION

On one GPU, under the same expert-memory budget, DynaByte reduces p95 TPOT by 11.6%, 23%, and 27.1% relative to HOBBIT on Qwen3-30B, Qwen3-Next-80B, and Phi-3.5-MoE. The lowest p95 centers in Table 5 are 43.7 ms, 117 ms, and 51.8 ms. The rows at those centers have ΔPPL above $\epsilon = 0.70$, 0.60, and 0.90. Its ΔPPL on the mixed evaluation prompts is 0.62, 0.52, and 0.82. These lie within $\epsilon = 0.70$, 0.60, and 0.90. Weighted by item count, its averages are 73.49, 79.04, and 60.73 against 74.37, 79.55, and 61.36, lower by 0.88, 0.51, and 0.63 points (1.18%, 0.64%, and 1.03%).

At equal or better ΔPPL, the gaps to Static-Joint are 2.8 ms and 27 ms on Qwen3-30B and Qwen3-Next-80B. On Qwen3-30B the gap is 0.4 ms at 40 GB/s, and at $\Phi = 1.931$ both centers are 43.8 ms. On Phi-3.5-MoE the 5.2 ms difference is 53.8 ms against 59.0 ms, at ΔPPL 0.82 against 0.76. Both values are within $\epsilon = 0.90$, and DynaByte does not dominate Static-Joint. Uniform-low is at ΔPPL 0.97 and 51.8 ms, above $\epsilon = 0.90$. Demoting or evicting experts inside the one shared budget accounts for the 2.8 ms and 27 ms gaps. Scoring the requested copy and the released expert separately leaves the center 0.3 ms and 3 ms lower, inside DynaByte's interval. At $\Phi = 0.7$, every policy has ΔPPL = 0.88, above $\epsilon = 0.70$. Under Poisson arrivals from a client on the same host, SLO goodput is 0.82 requests/s against HOBBIT's 0.69 on Qwen3-30B. The runtime is one GPU with host memory as the only backing store.

REFERENCES

[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation*. 117–134. https://www.usenix.org/conference/osdi24/presentation/agrawal

[2] Shiyi Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E. Gonzalez, Matei Zaharia, and Ion Stoica. 2025. MoE-Lightning: High-Throughput MoE Inference on Memory-constrained GPUs. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Vol. 1. 715–730. https://doi.org/10.1145/3669940.3707267

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] https://arxiv.org/abs/2107.03374

[4] Krishna Teja Chitty-Venkata, Jie Ye, and Murali Emani. 2025. MoPEQ: Mixture of Mixed Precision Quantized Experts. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*. https://arxiv.org/abs/2509.02512

[5] Mohammed Nowaz Rabbani Chowdhury, Kaoutar El Maghraoui, Hsinyu Tsai, Naigang Wang, Geoffrey W. Burr, Liu Liu, and Meng Wang. 2026. Efficient Quantization of Mixture-of-Experts with Theoretical Generalization Guarantees. In *The Fourteenth International Conference on Learning Representations*. https://arxiv.org/abs/2604.06515

[6] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168 [cs.LG] https://arxiv.org/abs/2110.14168

[7] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. arXiv:2401.06066 [cs.CL] https://arxiv.org/abs/2401.06066

[8] DeepSeek-AI, Aixin Liu, Bei Feng, Bin Wang, et al. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv:2405.04434 [cs.CL] https://arxiv.org/abs/2405.04434

[9] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*, Vol. 35. 30318–30332. https://doi.org/10.52202/068431-2198

[10] Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, et al. 2022. GLaM: Efficient Scaling of Language Models with Mixture-of-Experts. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*. 5547–5569. https://proceedings.mlr.press/v162/du22c.html

[11] Zhixu Du, Shiyu Li, Yuhao Wu, Xiangyu Jiang, Jingwei Sun, Qilin Zheng, Yongkai Wu, Ang Li, Hai H. Li, and Yiran Chen. 2024. SiDA: Sparsity-Inspired Data-Aware Serving for Efficient and Scalable Large Mixture-of-Experts Models. In *Proceedings of Machine Learning and Systems*, Vol. 6. 224–238. https://proceedings.mlsys.org/paper_files/paper/2024/hash/698cfaf72a208aef2e78bcac55b74328-Abstract-Conference.html

[12] Haojie Duanmu, Xiuhong Li, Zhihang Yuan, Size Zheng, Jiangfei Duan, Xingcheng Zhang, and Dahua Lin. 2025. MxMoE: Mixed-precision Quantization for MoE with Accuracy and Performance Co-Design. arXiv:2505.05799 [cs.LG] https://arxiv.org/abs/2505.05799

[13] Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. 2024. Extreme Compression of Large Language Models via Additive Quantization. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. 12284–12303. https://proceedings.mlr.press/v235/egiazarian24a.html

[14] Artyom Eliseev and Denis Mazur. 2023. Fast Inference of Mixture-of-Experts Language Models with Offloading. arXiv:2312.17238 [cs.LG] https://arxiv.org/abs/2312.17238

[15] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39. https://jmlr.org/papers/v23/21-0998.html

[16] Elias Frantar and Dan Alistarh. 2023. QMoE: Practical Sub-1-Bit Compression of Trillion-Parameter Models. arXiv:2310.16795 [cs.LG] https://arxiv.org/abs/2310.16795

[17] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In *The Eleventh International Conference on Learning Representations*. https://arxiv.org/abs/2210.17323

[18] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. 2023. MegaBlocks: Efficient Sparse Training with Mixture-of-Experts. In *Proceedings of Machine Learning and Systems*, Vol. 5. 288–304.

[19] Xin He, Shunkang Zhang, Kaijie Tang, Shaohuai Shi, Yuxin Wang, Zihao Zeng, Zhenheng Tang, Xiaowen Chu, Haiyan Yin, Ivor W. Tsang, and Yew Soon Ong. 2024. ExpertFlow: Efficient Mixture-of-Experts Inference via Predictive Expert

Caching and Token Scheduling. arXiv:2410.17954 [cs.LG] https://arxiv.org/abs/2410.17954

[20] Yuegui Huang, Zhiyuan Fang, Weiqi Luo, Ruoyu Wu, Wuhui Chen, and Zibin Zheng. 2026. DyMoE: Dynamic Expert Orchestration with Mixed-Precision Quantization for Efficient MoE Inference on Edge. arXiv:2603.19172 [cs.LG] https://arxiv.org/abs/2603.19172

[21] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, HoYuen Chau, Peng Cheng, Fan Yang, Mao Yang, and Yongqiang Xiong. 2023. Tutel: Adaptive Mixture-of-Experts at Scale. In *Proceedings of Machine Learning and Systems*, Vol. 5. 269–287.

[22] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. 2024. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 1018–1031. https://doi.org/10.1109/ISCA59077.2024.00078

[23] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of Experts. arXiv:2401.04088 [cs.LG] https://arxiv.org/abs/2401.04088

[24] Keisuke Kamahori, Tian Tang, Yile Gu, Kan Zhu, and Baris Kasikci. 2025. Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models. In *The Thirteenth International Conference on Learning Representations*. https://arxiv.org/abs/2402.07033

[25] Sehoon Kim, Coleman Richard Charles Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W. Mahoney, and Kurt Keutzer. 2024. SqueezeLLM: Dense-and-Sparse Quantization. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. 23901–23923. https://proceedings.mlr.press/v235/kim24f.html

[26] Young Jin Kim, Raffy Fahim, and Hany Hassan Awadalla. 2023. Mixture of Quantized Experts (MoQE): Complementary Effect of Low-bit Quantization and Robustness. arXiv:2310.02410 [cs.LG] https://arxiv.org/abs/2310.02410

[27] Rui Kong, Yuanchun Li, Qingtian Feng, Weijun Wang, Xiaozhou Ye, Ye Ouyang, Linghe Kong, and Yunxin Liu. 2024. SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6710–6720. https://doi.org/10.18653/v1/2024.acl-long.363

[28] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626. https://doi.org/10.1145/3600006.3613165

[29] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations*.

[30] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-Aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems*, Vol. 6. 87–100. https://proceedings.mlsys.org/paper_files/paper/2024/hash/42a452cbafa9dd64e9ba4aa95cc1ef21-Abstract-Conference.html

[31] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*, Vol. 32. 8026–8037. https://papers.nips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

[32] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 118–132. https://doi.org/10.1109/ISCA59077.2024.00019

[33] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*. 18332–18346. https://proceedings.mlr.press/v162/rajbhandari22a.html

[34] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–14. https://doi.org/10.1145/3458817.3476205

[35] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. 2024. GPQA: A graduate-level Google-proof Q&A benchmark. In *First Conference on Language Modeling*. https://arxiv.org/abs/2311.12022

[36] Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. 2024. OmniQuant: Omnidirectionally calibrated quantization for large language models. In *International Conference on Learning Representations*.

[37] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*.

[38] Zixu Shen, Kexin Chu, Yifan Zhang, Dawei Xiang, Runxin Wu, and Wei Zhang. 2025. ExpertFlow: Adaptive Expert Scheduling and Memory Coordination for Efficient MoE Inference. arXiv:2510.26730 [cs.PF] https://arxiv.org/abs/2510.26730

[39] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. 31094–31116. https://proceedings.mlr.press/v202/sheng23a.html

[40] Xiaoniu Song, Zihang Zhong, Rong Chen, and Haibo Chen. 2024. ProMoE: Fast MoE-based LLM Serving using Proactive Caching. arXiv:2410.22134 [cs.LG] https://arxiv.org/abs/2410.22134

[41] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2024. PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 590–606. https://doi.org/10.1145/3694715.3695964

[42] Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. 2024. HOBBIT: A Mixed Precision Expert Offloading System for Fast MoE Inference. arXiv:2411.01433 [cs.LG] https://arxiv.org/abs/2411.01433

[43] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, et al. 2024. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. In *Advances in Neural Information Processing Systems*, Vol. 37. 95266–95290. https://doi.org/10.52202/079017-3018

[44] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*. 38–45. https://aclanthology.org/2020.emnlp-demos.6/

[45] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *International conference on machine learning*. 38087–38099. https://proceedings.mlr.press/v202/xiao23c.html

[46] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. 2024. MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache. arXiv:2401.14361 [cs.LG] https://arxiv.org/abs/2401.14361

[47] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

[48] Jinghan Yao, Quentin Anthony, Aamir Shafi, Hari Subramoni, and Dhabaleswar K. Panda. 2024. Exploiting inter-layer expert affinity for accelerating mixture-of-experts model inference. In *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 915–925. https://doi.org/10.1109/IPDPS57955.2024.00086

[49] Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li, and Yuxiong He. 2022. ZeroQuant: Efficient and Affordable Post-Training Quantization for Large-Scale Transformers. In *Advances in Neural Information Processing Systems*, Vol. 35. 27168–27183. https://doi.org/10.52202/068431-1970

[50] Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. 2025. EdgeMoE: Empowering Sparse Large Language Models on Mobile Devices. *IEEE Transactions on Mobile Computing* 24, 8 (2025), 7059–7073. https://doi.org/10.1109/TMC.2025.3546466

[51] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation*. 521–538. https://www.usenix.org/conference/osdi22/presentation/yu

[52] Yifan Zhang and Math-AI Team. 2025. American Invitational Mathematics Examination (AIME) 2025. Hugging Face dataset. https://huggingface.co/datasets/math-ai/aime25

[53] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*, Vol. 37. 62557–62583. https://doi.org/10.52202/079017-2000

[54] Shuzhang Zhong, Ling Liang, Yuan Wang, Runsheng Wang, Ru Huang, and Meng Li. 2024. AdapMoE: Adaptive sensitivity-based expert gating and management for efficient MoE inference. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9. https://doi.org/10.1145/3676536.3676741

[55] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*. 193–210. https://www.usenix.org/conference/osdi24/presentation/

zhong-yinmin

[56] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M. Dai, Zhifeng Chen, Quoc V. Le, and James Laudon. 2022. Mixture-of-Experts with Expert Choice Routing. In *Advances in Neural Information Processing Systems*, Vol. 35. 7103–7114. https://doi.org/10.52202/068431-0515

Algorithm 1 One DynaByte control event.

Require: snapshot, published and reserved leases, envelope S_t

if prefetch hit rate < 0.25 **then**
  $S_t \leftarrow \max(S_t - 1, S_{\min})$
else if prefetch hit rate > 0.60 and a demand miss is outstanding **then**
  $S_t \leftarrow \min(S_t + 1, S_{\max})$
end if
$Z \leftarrow$ residency and lookahead leases with depth $\le S_t$
if this event is a slow boundary **then**
  add fidelity leases to Z
end if
if $\widehat{R}(X) > \widehat{\epsilon}/\kappa$ **then**
  close promotions of minimum replayed cost per unit of $\widehat{R}$
  admit them until $\widehat{R}(X) \le \widehat{\epsilon}/\kappa$
end if
for all lease $z \in Z$ **do**
  attach donors D in increasing counterfactual loss
  close $x = (z, D, \mathcal{T})$
  if x violates $\widehat{R} \le \widehat{\epsilon}/\kappa$, Eq. (7), or deadlines **then**
    discard x
  else
    compute $G(x \mid X)$ and $\eta(x \mid X)$ by deadline replay
  end if
end for
while some feasible x has $G(x \mid X) > 0$ **do**
  admit $x^\star \leftarrow \arg\max_x \eta(x \mid X)$
  apply $x^\star$ to provisional X and both accounts
  drop conflicting exchanges and replay the rest
end while
if this event is a slow boundary and the candidate would change the published fidelity set and the 100-step fidelity tenure has ended **then**
  reserve the candidate extents inside B and copy them on the serving GPU
  if the reservation would exceed B **then**
    discard the schedule and leave the published state unchanged
  else
    score the side probe on provisional X
    if the probe measures $\Delta\text{PPL} > \epsilon$ **then**
      roll back the last admitted exchange
      promote until the probe is within ϵ or no promotion fits
      if the probe still exceeds ϵ **then**
        discard the schedule and leave the published state unchanged
      else
        submit the admitted schedules $(Z^+, D, \mathcal{T})$
      end if
    else
      submit the admitted schedules $(Z^+, D, \mathcal{T})$
    end if
  end if
else
  submit the admitted schedules $(Z^+, D, \mathcal{T})$
end if